



TP 1 IA PARAMETRAGE DE VSCODE ET PYTHON

Python est un langage de programmation interprété contrairement à C/C++ qui lui est un langage de bas niveau compilé. Python est donc un moteur qui comprend une certaine syntaxe pour exécuter du code. Certaines bibliothèques Python sont codées en C++ ce qui permet d'être un langage facile à appréhender mais aussi très puissant.

Prérequis : Avoir un VM à jour `apt-get update / apt-get upgrade`, VS code sur son PC .

Se connecter sur sa VM via SSH de vsCode

1 Le moteur python doit donc être installé sur sa VM

- 1) Installé la dernière version de python sur votre VM .

Pour vous aider voici une commande qui permet de savoir ce qui est installable :

```
#apt policy python3
```

Une fois la version trouvée il faut l'installer avec la commande :

```
#apt install pythonX.xx ( ceci est un exemple )
```

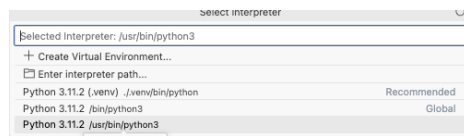
- 2) Quel est le retour exact du terminal Linux quand vous exécutez la commande suivante :

Commande 1 : `#python3 -c "print('julien code')"`

-
- 3) puis celle la Commande 2 : `#python3 -c "printd(julien code)"`
-

- 4) Installer sur VS Code l'extension officielle Microsoft pour Python pour chaque VM

Sous vsCode Appui sur F1 : "Python: Select Interpreter" et choisi le python de ta VM (usr/bin/ ect)



ne pas prendre .venv

Puis F1 → "Reload Window"

2 Installer les librairie suivantes sur chaque'une de vos VM:

Attention sous debian 12 il faut passer par apt et pas pip

 Bibliothèque	 Installation	 dites à quoi elle servent
numpy	apt install python3-numpy	_____
matplotlib	apt install python3-matplotlib	_____
pandas	apt install python3-pandas	_____
scikit-learn	apt install python3-sklearn	_____
jupyter	apt install jupyter-notebook	_____
seaborn (opt)	apt install python3-seaborn	_____

3 tester le debugger: 🟡

Voici un code python pour vérifier que tout fonctionne :

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.linear_model import LogisticRegression
```

```
print("✅ Tous les modules sont importés avec succès par mettez votre nom !")
```

```
# Mini test : graphique seaborn
```

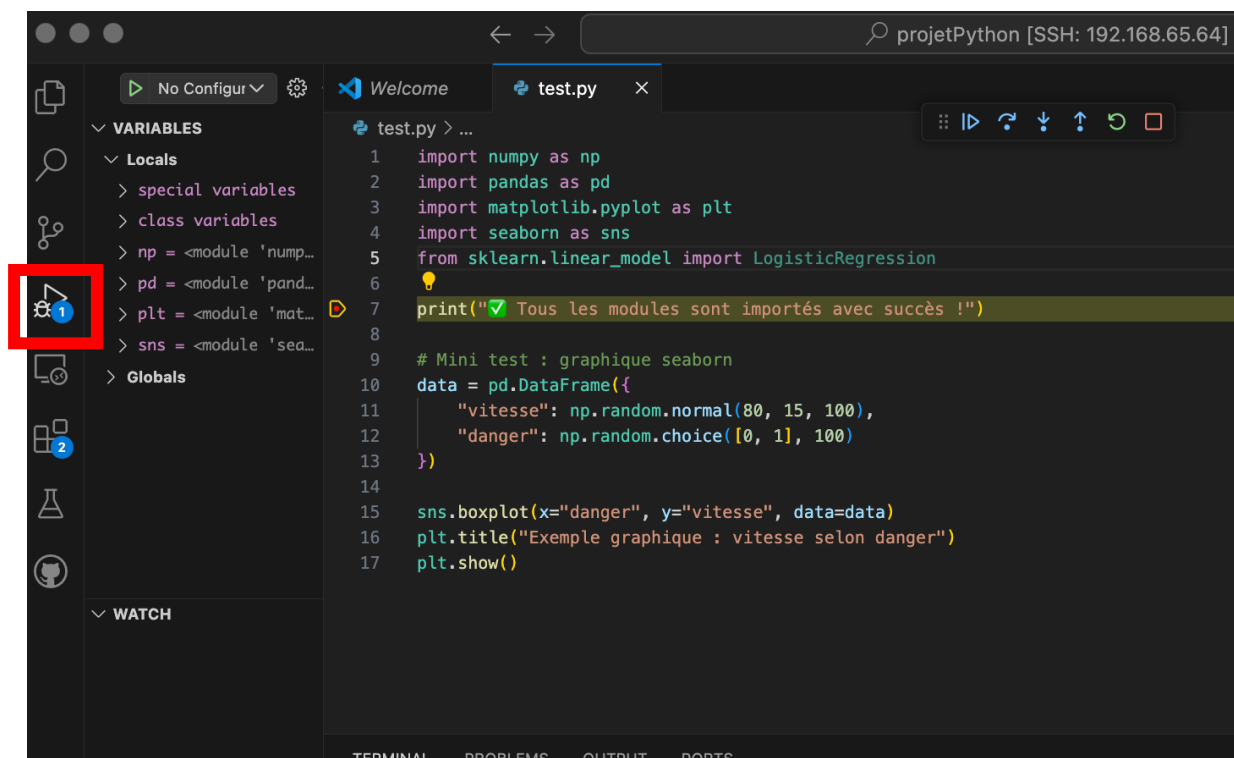
```
data = pd.DataFrame({  
    "vitesse": np.random.normal(80, 15, 100),  
    "danger": np.random.choice([0, 1], 100)  
})
```

```
sns.boxplot(x="danger", y="vitesse", data=data)
```

```
plt.title("Exemple graphique : vitesse selon danger")
```

```
plt.show()
```

- 1) Test ce code en mode debug et mettez un point d'arrêt pour vérifier que vous maîtriser le debugger



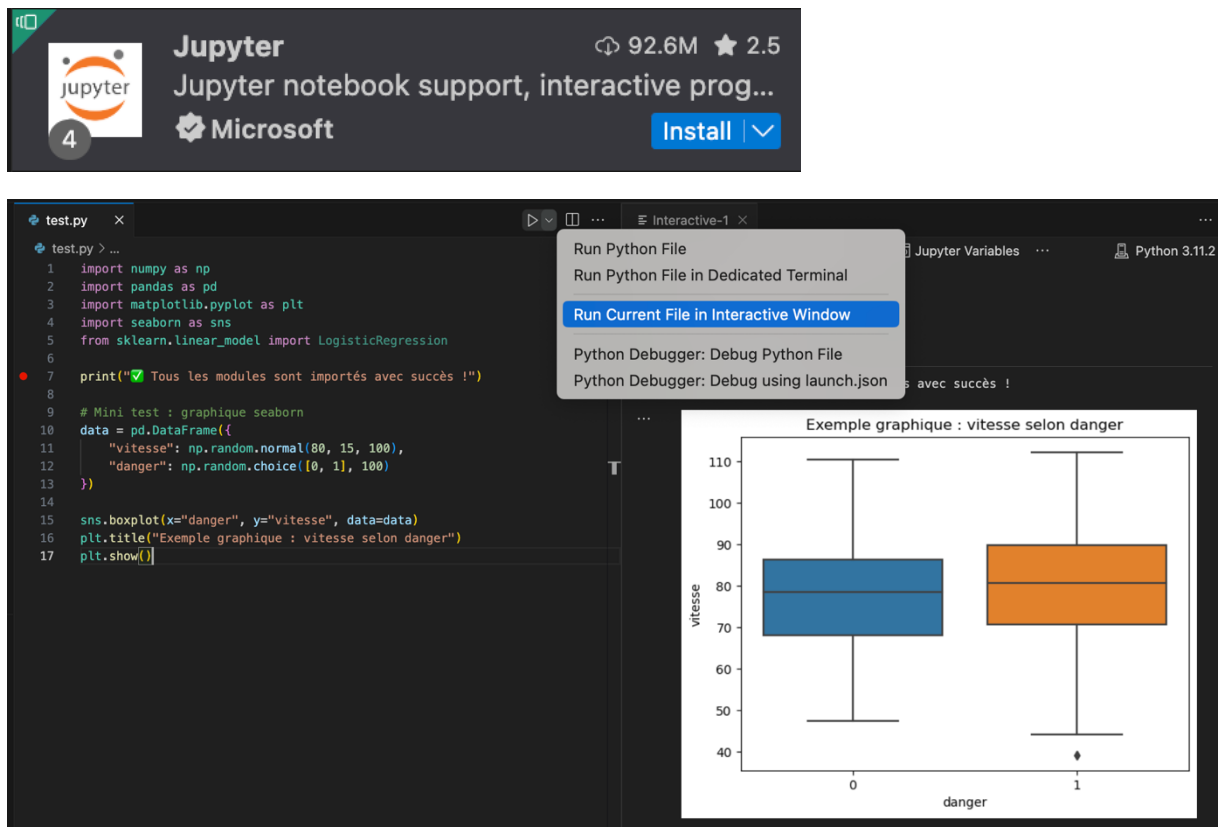
Si vous avez la barre jaune vous êtes bon.

- 2) Par contre il manque encore des choses car pour le moment on ne voit pas le plt show avec le title ect. Savez-vous pourquoi sans demander à IA ?

On va passer par un autre moyen que le terminal pour avoir les résultats visuels de python.

4 interface graphique déporté

- 1) installé le plugin vscode jupyter aussi pour chaque VM (oui une extension est valide en local mais aussi à distance



The screenshot shows the VS Code interface with the Jupyter extension installed. The extension's marketplace entry is visible at the top, showing it has 92.6M downloads and a 2.5 star rating. Below, the code editor shows a Python script named 'test.py' that imports numpy, pandas, matplotlib, and seaborn, and generates a box plot. A context menu is open over the code, with 'Run Current File in Interactive Window' selected. To the right, the interactive window displays the resulting box plot titled 'Exemple graphique : vitesse selon danger'. The plot shows two box plots: a blue one for 'danger' 0 and an orange one for 'danger' 1. The y-axis is labeled 'vitesse' and ranges from 40 to 110.

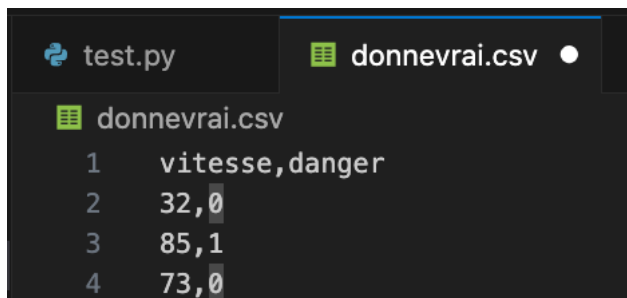
```
test.py > ...
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.linear_model import LogisticRegression
6
7 print("✅ Tous les modules sont importés avec succès !")
8
9 # Mini test : graphique seaborn
10 data = pd.DataFrame({
11     "vitesse": np.random.normal(80, 15, 100),
12     "danger": np.random.choice([0, 1], 100)
13 })
14
15 sns.boxplot(x="danger", y="vitesse", data=data)
16 plt.title("Exemple graphique : vitesse selon danger")
17 plt.show()
```

En lançant l'interactive windows vous avez les résultats de plt.show sous forme de graph

- 2) Changer le 0 par « Safe » et 1 par « danger » dans le code et relancer le test
- 3) En analysant le graphique obtenu. Que représente ses données ? (vous pouvez vous aider de l'IA pour comprendre ces boites bizarre bleu et orange.) _____
- 4) Pourquoi les vitesses dangereuse et vitesse Safe se mélange ? _____

4 Intégrer un jeu de donnée réel

- 1) Créer un fichier donnevrai.csv dans votre projet.



```
test.py  donnevrai.csv
donnevrai.csv
1  vitesse,danger
2  32,0
3  85,1
4  73,0
```

Il faut mettre vos données obtenues dans le TD 1 IA , la vitesse **virgule** 0 si safe et 1 si danger

- 2) Que veut dire CSV en français ? _____

Et oui si l'on souhaite mettre des chiffres à virgule il faut utiliser le point. 32.3 kmh c'est safe
32.3,0.

- 3) Dans votre code python rajouter sans l'aide de IA svp les données reel : avec le code suivant :

`df = pd.read_csv("votreFichier.csv")` // pour info pd est l'instance de pandas.

df sera votre variable qui remplace le DataFrame d'avant.

Donc maintenant veuillez charger vos données dans le paramètre data de la méthode boxplot

- 4) Observez maintenant vos graphiques. Vous semble-t-il plus judicieux ? expliquer et donnée votre avis ? que faire ressortir ce graphique ?

- 5) Pourquoi avoir utiliser DataFrame , a quoi il servait ?

Ceci termine notre introduction à python et ses outils. Il existe plein d'autre outil que l'on verra mais pour le moment nous n'avons pas encore entraîné notre neurone. On avons juste appris à contrôler notre jeu de donnée réel pour qu'il soit cohérent.